Software Requirements Specification: Personal Digital Archive

1. Introduction

1.1 Purpose

This document specifies the requirements for the Personal Digital Archive, a DBMS course project that uses a relational database (SQL) together with a vector database.

1.2 Scope

A local, single-user application (SUA) to store, organize, and retrieve personal files using SQL queries and semantic (vector) search.

1.3 Definitions

- Chunk: a segment of extracted text from an item.
- Embedding: a numeric vector representing the meaning of a chunk.
- Hybrid search: SQL filters combined with vector similarity.
- RBAC: role-based access control.
- FTS: full-text search.
- OCR: optical character recognition.

1.4 References

- Course syllabus
- PostgreSQL documentation
- Qdrant or Chroma documentation
- Apache Tika, MarkItDown, and Tesseract documentation

2. Overall Description

2.1 System context

- Next.js web UI
- PostgreSQL as the source of truth
- Vector DB storing only vectors and chunk_id
- Extraction layer (text extraction and OCR)
- Embedding worker

2.2 Actors

- User
- Embedding Worker (system)
- Admin (database level)

2.3 Supported formats

TXT/MD, PDF, DOCX, PPTX, XLSX/CSV, HTML, PNG/JPG. Other formats are stored with metadata only, with no content search.

2.4 Assumptions and constraints

- Local-only, one user
- One embedding model with a fixed vector dimension
- A few thousand items
- Extraction and OCR are delegated to existing tools

3. Functional Requirements

Item management

- FR1: Add, view, edit, and delete items.
- FR2: Each item stores title, description, mime_type/format, source, creation date, date added, and file path. The binary file stays on disk.
- FR3: Mark items as favorite or archived.

Extraction and content

- FR4: Extract plain text from supported formats, using OCR for images and scanned PDFs.
- FR5: Track extraction_status (pending, done, failed, unsupported) and the method used (text layer or OCR).
- FR6: Split extracted text into chunks.

Organization

- FR7: Collections are nestable (folder and subfolder). An item can belong to many collections.
- FR8: Tags are optional and free-form (many-to-many with items).

Embeddings

- FR9: Generate one embedding per chunk, store the vector in the vector DB, and record the model used.
- FR10: Track embedding_status per chunk (pending, done, failed).

Search

- FR11: Filter by format, tag, collection, and date range.
- FR12: Full-text keyword search over title, description, and extracted text.
- FR13: Semantic search: embed the query, take the top-k similar chunks, and map them back to their items.
- FR14: Hybrid search: SQL filters combined with vector similarity.

Activity and audit

- FR15: Log item access and search queries (for recently viewed and history).
- FR16: Write an audit log entry when an item is deleted.

Database logic

- FR17: Report queries: recursive listing under a folder, most recently accessed items, top tags by usage, untagged items, items accessed more than average, and items in collection A but not in B.
- FR18: Views: recent_items, item_summary, and statistics views (counts per collection, tag, and format).
- FR19: Triggers for automatic updated_at refresh and the delete audit.
- FR20: Stored procedure add_item and function search_items(filters).

Transactions and concurrency

- FR21: Ingest is a single atomic transaction (item, chunks, status). The vector write happens after commit. On failure the chunk stays pending.
- FR22: Workers claim pending chunks using SELECT ... FOR UPDATE SKIP LOCKED so no chunk is processed twice.

UI (Next.js)

- FR23: Pages to add, browse, search, and filter items, and to view collections and tags.

4. Data Requirements

4.1 Entities

- Item
- FileFormat (lookup table)
- Collection
- Tag
- Chunk (weak entity of Item)
- Embedding record
- AccessLog
- SearchLog
- AuditLog

4.2 Relationships

- Item to Collection: many-to-many
- Item to Tag: many-to-many
- Collection to Collection: self-reference (parent)
- Item to Chunk: one-to-many
- Chunk to Embedding record: one-to-one

4.3 SQL and vector DB responsibility

- SQL holds all metadata, chunk text, logs, and statuses.
- The vector DB holds only vectors and chunk_id (optionally item_id for filtering).
- SQL is the source of truth. The vector DB can be rebuilt from it.

4.4 Constraints

- Primary keys and foreign keys on all tables
- UNIQUE on tag name
- CHECK on status values
- NOT NULL on required fields
- Cascade delete from Item to Chunks

5. Access Control (DB-level RBAC)

- archive_admin: owns the schema, runs DDL, can read audit_log.
- archive_app: SELECT, INSERT, UPDATE, DELETE on core tables; EXECUTE on add_item and search_items.
- embedding_worker: SELECT on chunk; UPDATE on embedding_status only.
- report_reader: SELECT on statistics views only.

There is no application-level login and no user table.

6. Non-Functional Requirements

- Consistency: deleting an item removes its chunks and its vectors.
- Integrity: keys, foreign keys, and constraints are enforced by the database.
- Performance: semantic search returns in about one second at a few thousand items. Indexes (such as GIN for FTS) are verified with EXPLAIN before and after.
- Simplicity: one embedding model, one machine.
- Security: least privilege through roles.

7. Design and Analysis Deliverables

- ER diagram and relational schema (ER-to-relational reduction)
- Functional dependencies per table and a 3NF/BCNF justification
- Three to four queries in both relational algebra and SQL
- EXPLAIN comparison for one optimized query
- Transaction and locking demonstration
- RBAC test showing a denied operation

8. Syllabus Traceability

Unit 1

- ER model and reduction to relational schema: Section 4, ER diagram

- Constraints and keys: Section 4.4
- Aggregates and grouping: FR18
- Relational algebra: Section 7

Unit 2

- Recursive CTE: FR7, FR17
- Window functions: FR17
- Full-text search: FR12
- Subqueries, set operations, NULLs: FR17
- Views and joins: FR18
- Triggers: FR16, FR19
- Functions and procedures: FR20

Unit 3

- Functional dependencies and normal forms: Section 7
- Query optimization: Sections 6 and 7

Unit 4

- Transactions: FR21
- Concurrency control and locking: FR22
- Vector databases and vector search with relational DBs: FR9, FR13, FR14
- Web application development: FR23
- Users and privileges (RBAC): Section 5

9. Out of Scope

- Multi-user support and application-level authentication
- Sharing and sync
- Cloud deployment
- Mobile app
- Versioning
- Neo4j and key-value stores
- ML training